Нелинейные динамические структуры данных

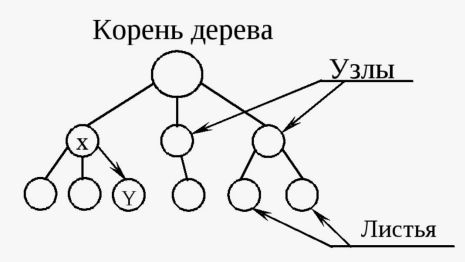
Дерево & Граф
Определения & Операции & Примеры

Автор: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей
МПГУ, 2026 | Язык примеров: C++

СДАП | Лабораторная работа 1

Нелинейные динамические структуры данных

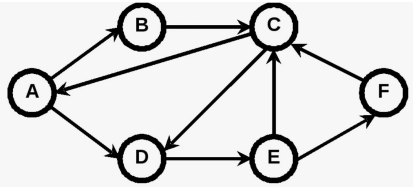
В отличие от линейных структур (массив, список, стек, очередь), элементы нелинейных структур могут иметь более одного предшественника или преемника. Они позволяют эффективно представлять иерархические и сетевые отношения.



Дерево

Иерархическая структура. Один корень, каждый узел имеет ноль или более потомков. Нет циклов.

Применение: файловые системы, DOM, бинарные деревья поиска, синтаксические деревья, кучи.



Граф

Множество вершин и рёбер (или дуг). Может содержать циклы.

Применение: сети, маршруты, социальные связи, зависимости задач, веб-граф, алгоритмы кратчайшего пути.

Дерево

В *теории графов* - связный неориентированный граф без циклов (или связный граф с минимальным числом рёбер). Число деревьев на *n* нумерованных вершинах = n^(n-2). Дерево с выделенной вершиной - **корневое**.

В *информатике* - иерархическая структура связанных узлов. Есть корень, от которого отходят ветви. Рекурсивная природа: каждое поддерево тоже является деревом. Сбалансированные деревья дают *O(log n)* для поиска/вставки/удаления.

Ключевые свойства и операции

- иерархия: один родитель –> ноль или более потомков;
- отсутствие циклов, рекурсивная структура, корневой узел;
- основные операции: Insert, Delete, Search, Traverse (pre-/in-/post-order, level-order);
- виды: бинарное, BST, AVL, красно-чёрное, куча (heap), B-дерево.

Пример

Узел бинарного дерева и вставка

```cpp
// Автор: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей
// Только одна операция: вставка в бинарное дерево поиска

#include <iostream>

struct Node {
    int key;
    Node* left;
    Node* right;
    Node(int k) : key(k), left(nullptr), right(nullptr) {}
};

Node* insert(Node* root, int key) {
    if (!root) return new Node(key);
    if (key < root->key)
        root->left = insert(root->left, key);
    else
        root->right = insert(root->right, key);
    return root;
}

int main() {
    Node* root = nullptr;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);

    std::cout << "Корень: " << root->key << "\n";
    std::cout << "Левый потомок: " << root->left->key << "\n";
    std::cout << "Правый потомок: " << root->right->key << "\n";
    return 0;
}
```

```
insert.cpp
:!g++ insert.cpp -o insert && ./insert && rm insert
```

```
Корень: 50
Левый потомок: 30
Правый потомок: 70
```

Пример

Обход бинарного дерева

```cpp
// Автор: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей
// Операция: in-order обход (сортировка BST)

#include <iostream>

struct Node {
    int key;
    Node* left;
    Node* right;
    Node(int k) : key(k), left(nullptr), right(nullptr) {}
};

Node* insert(Node* root, int key) {
    if (!root) return new Node(key);
    if (key < root->key)
        root->left = insert(root->left, key);
    else
        root->right = insert(root->right, key);
    return root;
}

void inorder(Node* n) {
    if (!n) return;
    inorder(n->left);
    std::cout << n->key << " ";
    inorder(n->right);
}

int main() {
    Node* root = nullptr;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);

    std::cout << "In-order: ";
    inorder(root);
    std::cout << "\n";
    return 0;
}
```

inOrder.cpp [+]

```
corgi@LAPTOP-AS2LVH3V:~/lab1$ g++ inOrder.cpp -o inOrder && ./inOrder && rm inOrder
In-order: 20 30 40 50 70
```

Пример

Поиск в бинарном дереве

```cpp
// Автор: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей
// Операция: поиск в бинарном дереве

#include <iostream>

struct Node {
    int key;
    Node* left;
    Node* right;
    Node(int k) : key(k), left(nullptr), right(nullptr) {}
};

Node* insert(Node* root, int key) {
    if (!root) return new Node(key);
    if (key < root->key)
        root->left = insert(root->left, key);
    else
        root->right = insert(root->right, key);
    return root;
}

Node* search(Node* root, int key) {
    if (!root || root->key == key)
        return root;
    if (key < root->key)
        return search(root->left, key);
    return search(root->right, key);
}
```

search.cpp

```cpp
}

int main() {
    Node* root = nullptr;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 40);

    Node* found = search(root, 40);
    if (found)
        std::cout << "Найден: " << found->key << "\n";
    else
        std::cout << "Не найден\n";

    found = search(root, 99);
    if (found)
        std::cout << "Найден: " << found->key << "\n";
    else
        std::cout << "Не найден\n";

    return 0;
}
```

search.cpp

```
corgi@LAPTOP-AS2LVH3V:~/lab1$ g++ search.cpp -o search && ./search && rm search
Найден: 40
Не найден
```

Граф

Граф G(V, E) - множество вершин V и множество рёбер E (неупорядоченных пар) или дуг (упорядоченных). **Неориентированный** - только рёбра; **ориентированный** - только дуги. Возможны *кратные* рёбра и *петли*.

Вершины, соединённые ребром/дугой - *смежные*. Ребро и его вершины - *инцидентны*. *Обход графа* - маршрут с заданными свойствами. *Расстояние* - длина кратчайшей простой цепи.

Неориентированный

Рёбра без направления. (u,v) = (v,u).
Примеры: соцсети, дорожные карты.

Ориентированный

Дуги с направлением. (u,v) ≠ (v,u).
Примеры: веб-ссылки, зависимости задач.

Взвешенный

Рёбра имеют вес.
Алгоритмы: алгоритмы Дейкстры, Прима, Краскала.

Пример

Добавление ребра в граф

```cpp
// Автор: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей
// Операция: добавление ребра в граф

#include <iostream>
#include <vector>

class Graph {
    int n;
    std::vector<std::vector<int>> adj;
public:
    Graph(int vertices) : n(vertices), adj(vertices) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);  // неориентированный
    }

    void print() {
        for (int i = 0; i < n; i++) {
            std::cout << i << ": ";
            for (int v : adj[i])
                std::cout << v << " ";
            std::cout << "\n";
        }
    }
};

int main() {
    Graph g(4);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);

    std::cout << "Списки смежности:\n";
    g.print();
    return 0;
}
```

addEdge.cpp [+]

```
corgi@LAPTOP-AS2LVH3V:~/lab1$ g++ addEdge.cpp -o addEdge && ./addEdge && rm addEdge
Списки смежности:
0: 1 2
1: 0 3
2: 0
3: 1
```

Пример

Обход графа в ширину

```cpp
// Автор: Студент группы ЗОМ34-ПВШ261 Золотухин Андрей Александрович
// Только одна операция: обход в ширину (BFS)

#include <iostream>
#include <vector>
#include <queue>

class Graph {
    int n;
    std::vector<std::vector<int>> adj;
public:
    Graph(int vertices) : n(vertices), adj(vertices) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void BFS(int start) {
        std::vector<bool> visited(n, false);
        std::queue<int> q;
        visited[start] = true;
        q.push(start);

        std::cout << "BFS: ";
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            std::cout << u << " ";
            for (int v : adj[u]) {
                if (!visited[v]) {
                    visited[v] = true;
                    q.push(v);
                }
            }
        }
        std::cout << "\n";
    }
};

int main() {
    Graph g(4);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(1, 3);

    g.BFS(0);
    return 0;
}
```

bfs.cpp [+]

```
corgi@LAPTOP-AS2LVH3V:~/lab1$ g++ bfs.cpp -o bfs && ./bfs && rm bfs
BFS: 0 1 2 3
```

Спасибо за внимание!